

Airflow Class Activity — Retail ETL Pipeline

Duration

45–60 Minutes

Objective

In this activity, you will design and implement a complete ETL pipeline using Apache Airflow.

You are expected to build and deploy an Airflow DAG using Astro CLI and Astronomer deployment access available in your VM.

This activity is intended to test your understanding of:

- Airflow DAG creation
- Task orchestration
- Scheduling
- Data validation
- ETL workflows
- Logging and debugging
- Retry handling
- Deployment workflows
- Airflow best practices

Scenario

You are working as a Data Engineer for a retail analytics company.

Every day, multiple stores upload sales transaction files. Your responsibility is to build an automated ETL pipeline that:

1. Extracts daily sales data
2. Validates incoming records
3. Cleans bad records
4. Transforms the data
5. Loads curated output data
6. Generates summary reports

The pipeline must run automatically every morning.

Environment Available

You already have access to:

- Ubuntu EC2 VM
- Astro CLI
- Astronomer authenticated session
- Airflow deployment
- DAG deployment access

Activity Instructions

You must build a complete Airflow DAG for the ETL workflow described below.

You are expected to create everything yourself.

Do not use prebuilt DAGs.

Step 1 — Create Project Structure

Inside your Astro project, create the following folders:

```
include/data/input/  
include/data/output/  
include/data/rejected/  
include/data/report/
```

Step 2 — Create Input Dataset

Create the following file inside:

```
include/data/input/
```

Filename:

```
orders_2026_05_08.csv
```

Step 3 — Add the Following CSV Data

```
order_id,store_id,product,category,quantity,price_per_unit,order_date,payment_mode  
1001,S001,Laptop,Electronics,2,55000,2026-05-08,Credit Card  
1002,S002,Headphones,Electronics,5,2000,2026-05-08,UPI  
1003,S001,Chair,Furniture,1,7000,2026-05-08,Cash  
1004,S003,Desk,Furniture,-1,12000,2026-05-08,Credit Card
```

1005,S002,Mouse,Electronics,10,800,2026-05-08,UPI
1006,S001,Keyboard,Electronics,0,1500,2026-05-08,Cash
1007,S004,Sofa,Furniture,1,25000,2026-05-08,Credit Card
1008,S003,Monitor,Electronics,3,15000,2026-05-08,UPI
1009,S002,Table,Furniture,2,8500,2026-05-08,Cash
1010,S005,Printer,Electronics,1,18000,2026-05-08,Credit Card

Business Rules

A record is considered invalid if:

- quantity ≤ 0
- price_per_unit ≤ 0
- payment_mode is empty

All invalid records must be written into a rejected file.

Required Transformations

For all valid records, add the following columns:

total_amount

quantity * price_per_unit

gst

18% of total_amount

final_amount

total_amount + gst

DAG Requirements

You must create separate Airflow tasks for each stage listed below.

Required Pipeline Flow

Task 1 — Check Input File Exists

Requirements:

- Verify the CSV file exists before processing
- If the file is missing, downstream tasks must not run

Task 2 — Extract Data

Requirements:

- Read CSV data
- Log the number of rows extracted
- Store metadata using XCom or another Airflow-supported mechanism

Task 3 — Validate Records

Requirements:

- Separate valid and invalid records
- Write rejected records into:

include/data/rejected/

Task 4 — Transform Data

Requirements:

Add the following columns:

- total_amount
- gst
- final_amount

Task 5 — Load Curated Data

Requirements:

Write transformed records into:

include/data/output/

Output filename:

processed_orders.csv

Task 6 — Generate Summary Report

Generate a report containing:

- Total valid orders
- Total rejected orders
- Total revenue
- Total GST collected
- Top-selling category

Store the report inside:

include/data/report/

Task 7 — Final Notification Task

Create a final task that logs:

Pipeline completed successfully

This task should run only if all critical tasks succeed.

DAG Configuration Requirements

Your DAG must include:

- owner
- retries
- retry_delay
- start_date
- schedule
- catchup=False

Scheduling Requirement

The DAG must run daily at:

6:00 AM

Suggested Project Structure

```
project/
├── dags/
│   └── retail_etl_dag.py
├── include/
│   └── data/
│       ├── input/
│       ├── output/
│       ├── rejected/
│       └── report/
```